Aman: An Anura manual

Anura is a GUI debugger with most simple and some advanced features to help you develop and debug your programs.

Contents

UI

Startup

Anura will open very blank, don't worry this just leaves room for your wonderful program to fit. To launch a new process use the file drop-down in the top left, and select 'open'. Anura can open any Linux-x64 ELF binary.

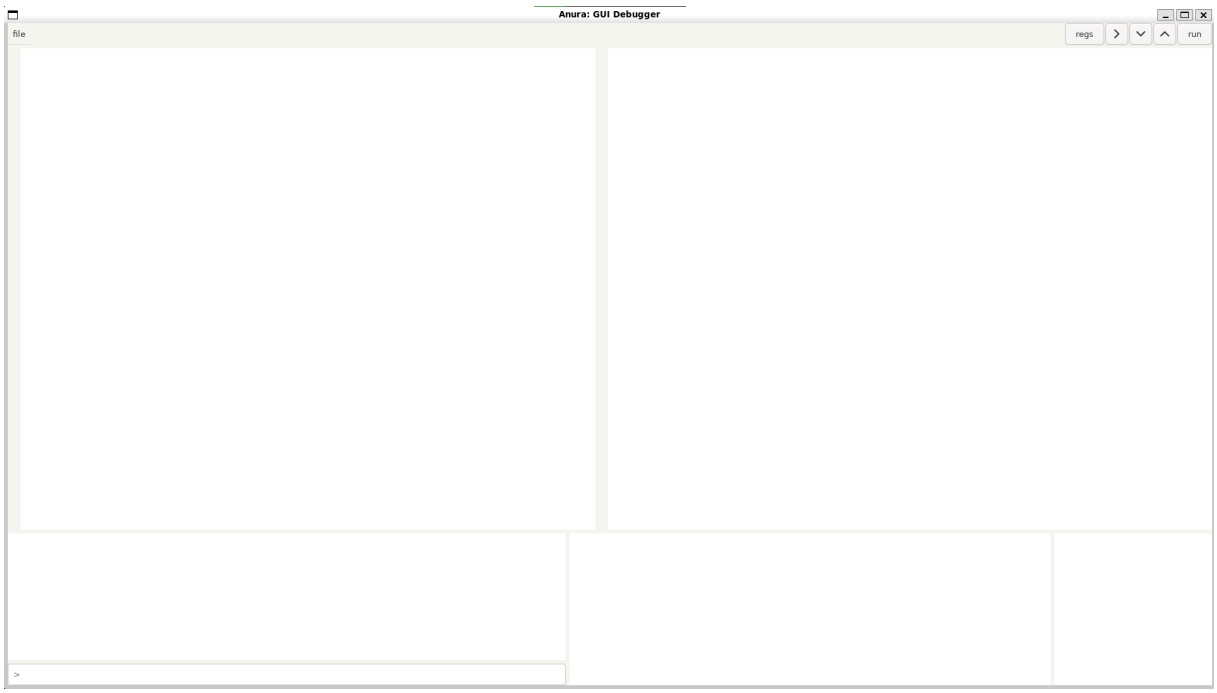

Figure 1: View on launch of Anura

Basic view

After opening a file much of the space will now have content. All the way on the left is the file explore dropper, this will contain a tree of the source files within your project. Next to this is the source code of the file that contains the main function, this scrollable view is not editable. On the right side is the disassembly of the current file, this follows Intel's syntax. Along side the disassembly is the current byte values at the addresses.

Below all of these are the terminals. The left terminal is the output of Anura, this includes logging information for parsing the files, this terminal also includes input which allows you to enter commands for Anura. The right terminal shows the output of your program as it executes, specifically the standard output.

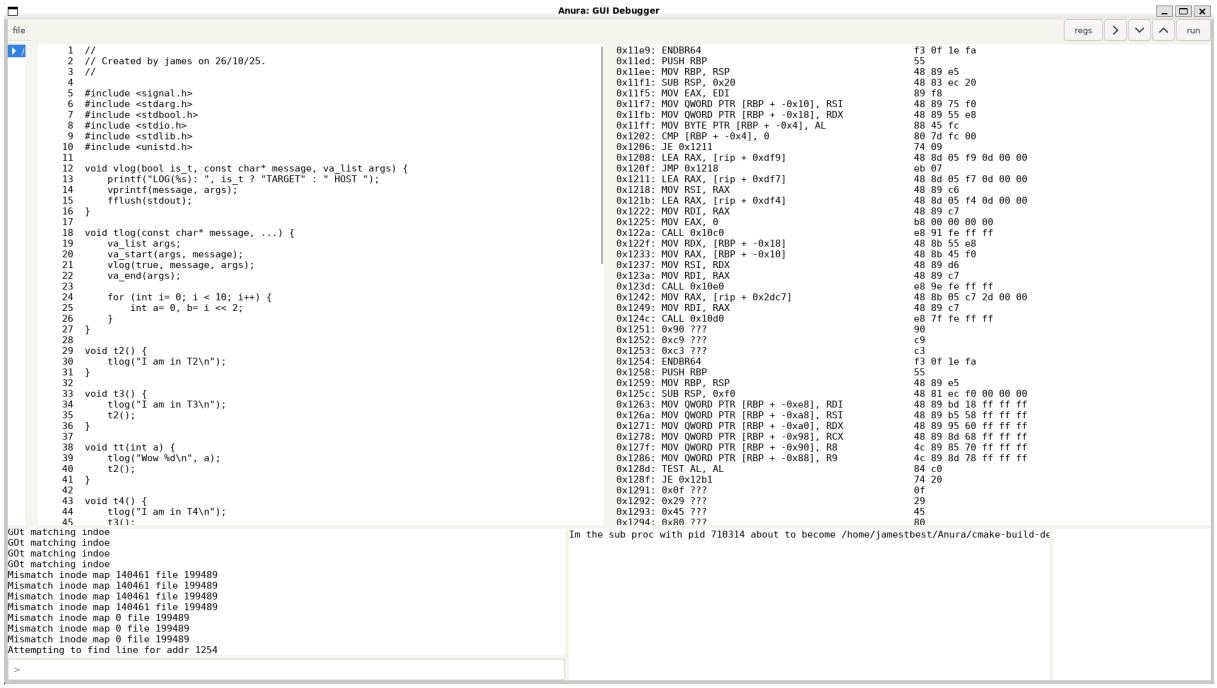

Figure 2: View after opening program

Breakpoints

The first interacting feature of Anura is placing breakpoints, this is done either through the terminal or by clicking in the gutter between the file view and source view. After adding a breakpoint a red circle will appear in both the source view on the line, and in the disassembly on the address.

Figure 3: View after placing breakpoint

Control flow

There are several features to allow you to skip through parts of the program's execution, these appear in the forms of step-over, step-into, step-out, and continue. These are found in that order in the top right, as a series of buttons.

Figure 4: Control flow options

Step-over
This function will skip over the current source line, ending at the next executed line in the current function. Any function calls within the current line are ignored.

Step-into
This function will enter the next function call and break at the start of it.

Step-out
This function will break on the return address of the current function, and so will break in the calling function.

Code 1 illustrates the different steps.

```c
int f() {
  // step-into destination
  return 1;
}

int g() {
  int a= 1;
  a += f();
  // Starting point
  return a;
  // step-over destination
}

int main() {
  int x= g();
  return x;
  // step-out destination
}
```

Code 1: Stepping destinations

Continue
Continue will return execution with no restrictions, the program will not break until it hits a breakpoint or a new signal is generated.

Break-save

Break save is a demo feature which currently only works on one program, BreakExample. Break save is designed to use compiler information to save stack frames, control flow, and data changes, to better understand why a variable evaluates to some value. Break-save is only accessible through the terminal with 'break save', and should be run from the program being in a stopped state on line 65.

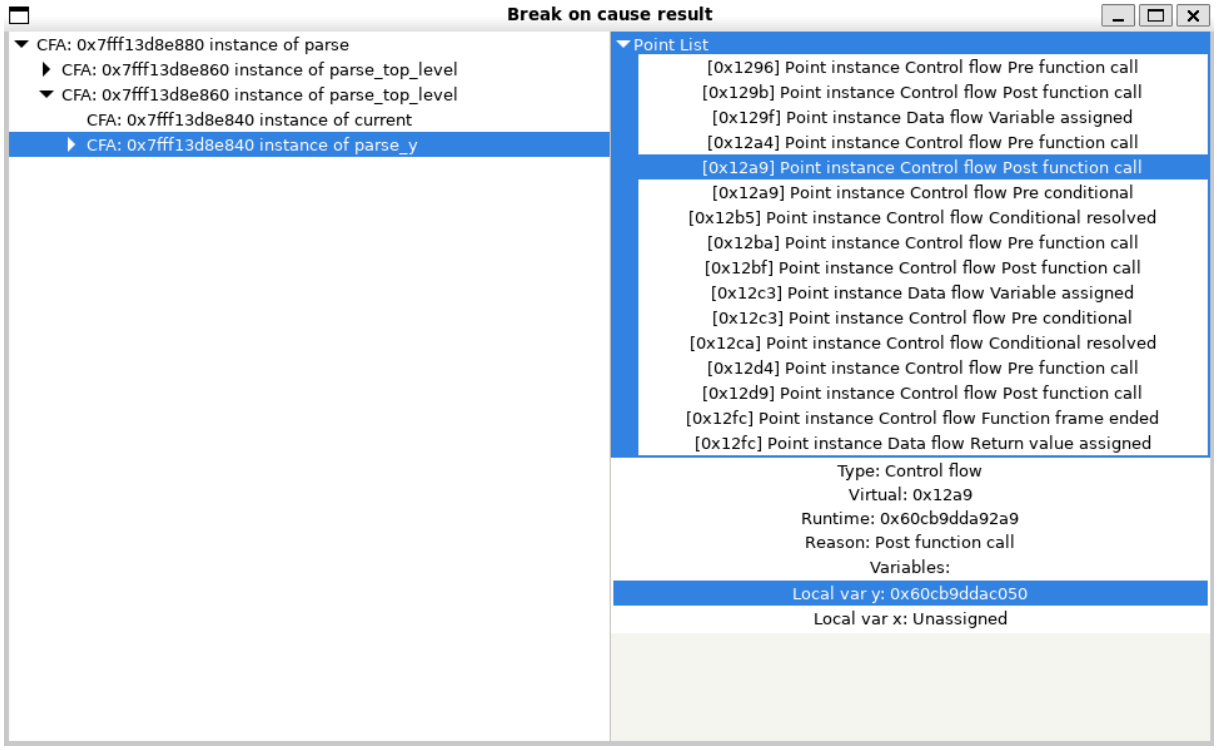

Figure 5: Break save output

The output of break save is a GUI window showing the stack frames that were saved on the left. The selected frame's control flow and data flow points are displayed on the right. The selected point's information is on the bottom right, along with the state of the variables at that point. Selecting a control flow point will highlight the area of code in the source view with a blue line.

Break-on-cause

Break-on-cause is similar to break-save in its goal, it is also designed to help understand why a variable evaluates to a certain value. However, break-on-cause does this at the earliest point of execution by using compiler information to propagate these target values throughout the program. Same as break-save this is a demo feature that only works on certain programs, specifically BreakExample and BreakSimple. It is also only available through the terminal, with 'break cause EXAMPLE|SIMPLE'.

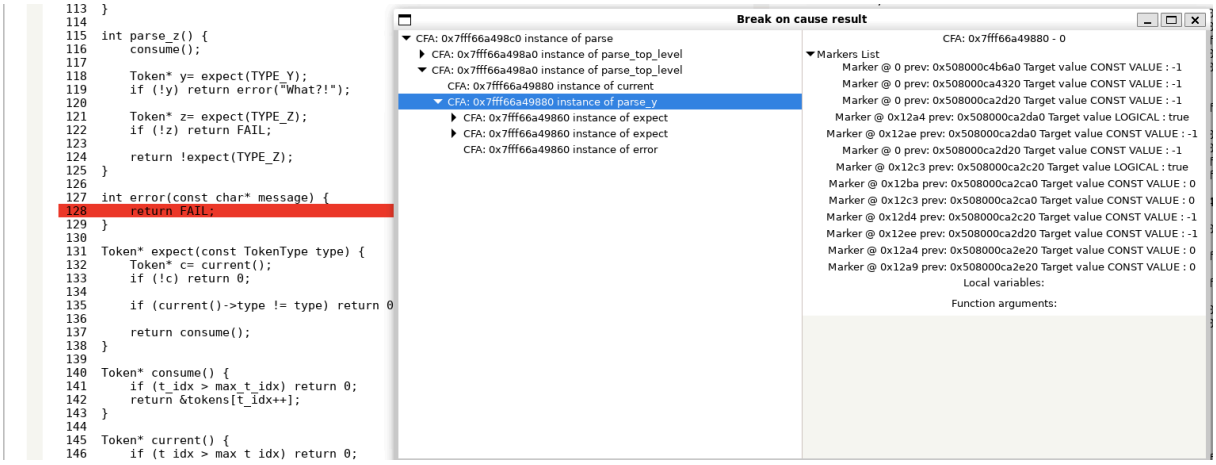

Figure 6: Break cause output

The output of break-on-cause is another GUI window, which similarly shows the saved stack frames on the left. In this case however, the program will have halted at the source of the error, in this case 'return FAIL'. On the right is a list of the markers for this frame, these are largely implementation information, these carry the target values to the different parts of the program. There is also a view of the variables at the end of the frame, and the function arguments.

Commands

Most commands should only be entered when your program is in a stopped state, these are marked in the table's X column. Stopped states are mainly brought about through breakpoints being hit, or signals being generated, a pesky SIGSEGV perhaps.

Command	Sub-command	inputs	description	X
breakpoint	set	<line>	Add a user breakpoint at the specified source line	X
	del	<line>	Remove a user breakpoint at the specified line	X
cf	cont		Continue execution of the program	X
	astep		Single step the program by one instruction	X
step	over		Step over the current source line, skipping all function calls	X
	into		Step into the current source line, entering any function calls	X
	out		Step out of the current source line, ending at the return address of the function	X
exit			End execution of the current process	
display	regs		Display the register view in the terminal	X
	stack		Display the stack trace in the terminal	X
	breakpoints		Display the currently placed user and internal breakpoints	
break	save		Run the break-save program	X
	cause	EXAMPLE/SIMPLE	Run the break-cause program, either with the compiler information for the example program or simple program	X